

10주차 스터디

교재 : 자연어 처리와 컴퓨터비전

분량 : 5,6장

과제 기간 : ~2025-05-12

5. 토큰화

단어 및 글자 토큰화

단어 토큰화

글자 토큰화

형태소 토큰화

형태소 어휘 사전

하위 단어 토큰화

BPE (Byte Pair Encoding)

워드피스(WordPiece)

6. 임베딩

언어 모델

자기회귀 언어 모델

통계적 언어 모델

N-gram 모델

TF-IDF

단어 빈도

문서 빈도

역문서 빈도

TF-IDF

5. 토큰화

- 자연어(Natural Language), 자연 언어
- 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어를 의미한다
- 자연어 처리(NLP) : 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
- 자연어 처리는 인공지능의 하위 분야 중 하나
- 컴퓨터가 인간의 언어를 이해하고 처리하는 것이 주요 목표
- 모델 개발을 위해 해결되어야 할 문제들

- 모호성(Ambiguity): 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 한다.
- 가변성(Variability): 인간의 언어는 다양한 사투리(Dialects), 강세(Accent), 신조어(Coined word), 작문 스타일로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 한다.
- 구조(Structure): 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하여 의미(Semantic)를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

- 말뭉치, Corpus : 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모 자연어
- 토큰 : 개별 단어나 문장 부호와 같은 텍스트 의미하며 말뭉치보다 더 작은 단위
- 토큰으로 나누는 목적은 컴퓨터가 자연어를 이해할 수 있게 나누는 과정을 **토큰화**라고 한다
- 토큰라이저를 구축하는 방법은 아래와 같다

- 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
- 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
- 어휘 사전(Vocabulary) 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
- 머신러닝 활용: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.

- 어휘사전 적용은 사전에 정의된 단어를 활용해 구축하는 방법이다
- 어휘사전 적용 방법은 직접 어휘 사전을 구축하므로 없는 단어나 토큰(OOV)가 존재할 수 없다
- 큰 어휘 사전을 구축하면 학습 비용이 증대되고 차원의 저주에 빠질 수도 있다

단어 및 글자 토큰화

- 토큰화는 NLP에서 매우 중요한 전처리 과정이다
- 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업이다
- 입력된 텍스트 데이터를 단어나 글자로 나누는 기법으로 단어 토큰화, 글자 토큰화가 있다

단어 토큰화

- Word Tokenization
- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화를 수행한다

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱',  
'최고!!']
```

- 구분자를 따로 입력하지 않으면 공백을 기준으로 나눈다
- 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류에 취약하다

글자 토큰화

- Character Tokenization
- 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식
- 비교적 작은 단어 사전을 구축할 수 있다
- 장점 : 컴퓨터 자원 아끼기, 말뭉치 학습 시 각 단어를 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용된다

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불', '여', ' ', 'o', 's', 't',  
' ', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

- 자소 단위로 나눠 자소 단위 토큰화를 수행해야한다 - 교재에서는 자모 라이브러리 활용

- 자모 라이브러리에서 사용하는 자모 변환 함수와 한글 호환성 자모 변환 함수를 이용해 입력된 한글 문자열을 한글 자모로 변환한다

자모 변환 함수

```
retval = jamo.h2j(
    hangul_string
)
```

한글 호환성 자모 변환 함수

```
retval = jamo.j2hcj(
    jamo
)
```

- 글자 단위로 토큰화하면 단어 단위로 토큰화하는 것에 비해 비교적 적은 크기의 단어 사전을 구축할 수 있다

예제 5.3 자소 단위 토큰화

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

출력 결과

```
['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄴ', ' ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅌ', 'ㄴ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅌ', 'ㄴ', ' ', 'ㄷ', 'ㄱ', 'ㅣ', ' ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅌ', 'ㄱ', 'ㅌ', 'ㄱ', ' ', 'ㅈ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅌ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ', ' ', 'ㅈ', 'ㅍ', 'ㄱ', 'ㅌ', '!', ' ', 'ㄷ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㅇ', 'ㅌ', ' ', 'ㅇ', 'ㅅ', 'ㅌ', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅌ', 'ㄷ', 'ㅌ', 'ㅇ', 'ㅌ', 'ㄱ', ' ', 'ㅈ', 'ㅍ', 'ㄱ', 'ㅌ', '!', '!']
```

- 접사와 문장 부호의 의미 학습이 가능하다
- 작은 크기의 단어 사전으로도 OOV를 획기적으로 줄일 수 있다
- 단 개체명 인식을 구현할 경우 도메인 구별이 어려울 수 있으며 시퀀스 길이가 길어질수록 연산량이 증가한다

형태소 토큰화

- Morpheme Tokenization
- 텍스트를 형태소 단위로 나누는 토큰화 방법
- 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업이다
- 한국어와 같이 교착어인 언어에서는 특히 중요하게 수행된다
- 문장 내 각 형태소 역할 파악, 문장 이해 등에 좋다

형태소 어휘 사전

- Morpheme Vocabulary
- 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전
- 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 정보가 제공된다
- 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업을 품사 태깅(POS Tagging)이라고 한다
- 자연어 처리 분야에서 문맥을 고려해 더 정확한 분석을 가능하게 한다

⇒ 이제 KoNLPy, NLTK, spaCY 라이브러리를 활용해 형태소 단위의 토큰화를 실습해보자

하위 단어 토큰화

- **형태소 분석**은 자연어 처리에서 중요한 전처리 과정으로, 단어를 의미 단위로 나누는 작업이다
- 그러나 **신조어, 고유어, 오타자** 등으로 인해 기존 형태소 분석 방식은 한계가 있다
- 이런 문제를 해결하기 위해 **하위 단위 토큰화(Subword Tokenization)** 방식이 사용된다

BPE (Byte Pair Encoding)

- BPE(Byte Pair Encoding)는 자주 등장하는 문자 쌍을 반복적으로 병합하는 방식이다

- 처음에는 모든 단어를 **문자 단위로 분해**한 후, 가장 자주 등장하는 쌍을 하나의 단위로 병합한다
- 예시: abracadabra → 'ab', 'ra', ... → 'AracadAra' → 'ABcadAB' → 최종적으로 'Ccacdc'
- 주어진 단어들의 빈도와 문자 쌍을 바탕으로 가장 자주 나타나는 쌍을 병합한다
- 예: 'lower', 'lower', 'newest', 'widest' → 'es' 병합 → 'est' 병합
- 이를 통해 어휘 사전에 새로운 단어가 없어도 유연하게 처리할 수 있다
- 신조어, 고유어, 오타자 등의 처리에 강하다
- 어휘 크기를 줄이면서도 OOV(Out-Of-Vocabulary) 문제를 완화한다
- 효율적인 토큰라이징을 통해 학습 성능을 향상시킬 수 있다

워드피스(WordPiece)

- 바이트 페어 인코딩과 유사한 방식의 **하위 단어 토큰라이저**
- 문자 기반으로 시작하여 **빈도 기반 + 확률 기반**으로 글자 쌍을 병합한다
- 모델이 학습 도중 얻은 **통계적 정보**를 바탕으로 글자 쌍을 병합한다
- 병합 시, **조건부 확률이 높은 쌍**을 우선적으로 선택해 단어로 확장한다
- 학습은 **더 이상 병합할 쌍이 없거나, 어휘 사전 크기에 도달할 때까지** 반복한다
- **빈도 기반 + 확률 기반 병합 방식**을 사용하여 더 정교한 토큰라이징이 가능하다
- 새로운 단어(OOV)가 들어와도 **기존 어휘 조각들**을 조합해 처리 가능하다
- BERT 등 많은 모델에서 사용되는 대표적인 토큰라이저 방식이다
- 글자 쌍 병합 점수 수식

$$score = \frac{f(x, y)}{f(x), f(y)}$$

기호	의미
$f(xy)f(xy)$	글자 쌍 $xyxy$ 의 등장 빈도
$f(x)f(x)$	글자 x 의 등장 빈도
$f(y)f(y)$	글자 y 의 등장 빈도
$score \text{ } \text{\texttt{score}}$	글자 쌍 $xyxy$ 가 함께 병합될 '적절성' 정도를 나타내는 점수

- ex)
 - '하'와 '는'이 각각 자주 등장하지만 함께는 드물다면 → 낮은 score
 - '하'와 '다'는 함께 등장하는 경우가 많으면 → 높은 score → 병합

6. 임베딩

- 텍스트 벡터화(Text Vectorization)는 문장을 숫자로 변환하는 과정으로, 컴퓨터가 텍스트를 이해할 수 없기 때문에 필요한 처리 단계이다.
- 대표적인 벡터화 방식으로는
 - 원-핫 인코딩(One-Hot Encoding)과
 - 빈도 벡터화(Count Vectorization)가 있다
- **원-핫 인코딩**은 각 단어를 고유한 색인으로 매핑한 후, 해당 색인 위치만 1로 표시하고 나머지는 0으로 표시하는 방식이다.
 - 예시: "I like apples", "I like bananas"를 원-핫 인코딩하면

- "I like apples": [1, 1, 1, 0]
- "I like bananas": [1, 1, 0, 1]로 표현된다.
- **빈도 벡터화**는 문서에서 단어의 등장 횟수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식이다.
 - 예시: "apples"가 2회 등장하면 해당 항목은 2로 표시된다
- 원-핫 인코딩의 단점
 - 구현이 간단하지만, 단어가 많아질수록 벡터가 매우 희소해지고(Sparsity), 공간 비효율성과 계산 낭비가 크다
 - 단어 간 의미 관계를 반영하지 못하며, "ice cream"과 "i scream"처럼 유사한 문장도 완전히 다르게 인코딩된다
- 원-핫 인코딩의 문제를 해결하기 위해 **워드 임베딩(Word Embedding)**이 등장했다
- **워드 임베딩**은 단어를 고정된 차원의 실수 벡터로 변환하는 기법으로, 의미가 비슷한 단어들은 공간상에서 가까운 벡터로 위치시키는 방식이다.
- 대표적인 방법으로는 **워드투벡(Word2Vec)**과 **패스트텍스트(fastText)** 등이 있다.
- 고정 임베딩이 아닌, 학습 중 의미 벡터를 계속 업데이트하는 방식은 ****동적 임베딩(Dynamic Embedding)****이라고 한다.

언어 모델

- Language Model
- 입력된 문장으로 다음 단어를 예측할 수 있는 확률을 계산하는 모델을 의미한다
- 문장의 구성이나 문법을 이해하고 문장 생성에 필요한 예측을 수행한다
- 음성 인식, 기계 번역, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용된다

- '안녕하세요'라는 문장 뒤에 어떤 문장이 올 확률을 계산하는 것이 언어 모델의 핵심이다

Input : 안녕하세요 ... Input : 대한민국 ___ 강남구 ...	
$P(\text{만나서 반갑습니다.}) = 0.081$	$P(\text{서울특별시}) = 0.59$
$P(\text{아니오, 괜찮습니다.}) = 0.000001$	$P(\text{아이스 아메리카노}) = 0.000001$

그림 6.1 언어 모델

자기회귀 언어 모델

- Autoregressive LM
- 입력된 단어들의 순서를 기반으로 다음 단어의 확률을 예측하는 모델
- 다음 단어를 예측하기 위해 이전까지 등장한 모든 단어 시퀀스를 고려하는 방식이다
- **조건부 확률의 연쇄 법칙(Chain rule)**을 활용하여 전체 문장의 확률을 각 단어의 조건부 확률 곱으로 계산한다

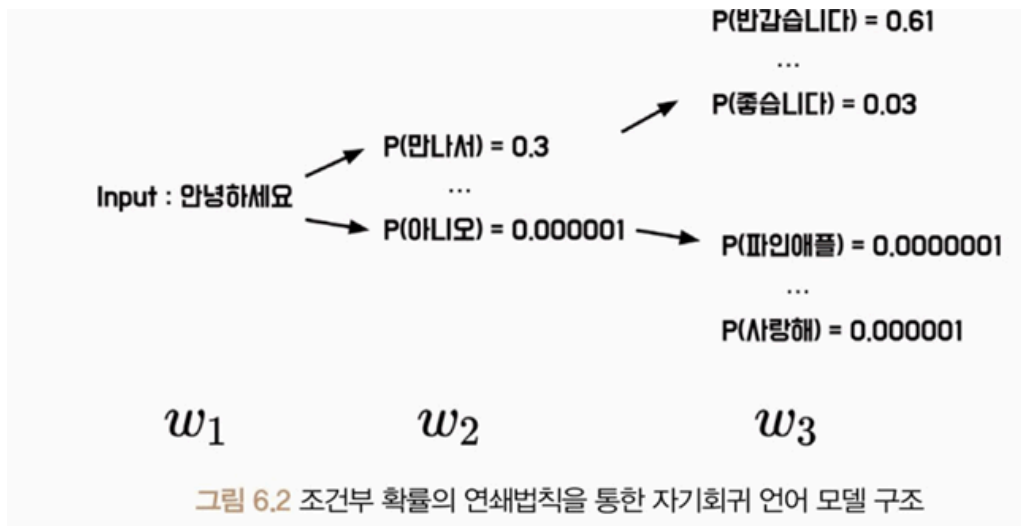
수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

- 다음 단어를 예측하고 이를 다시 모델의 입력으로 넣어 다음 단어를 예측하는 과정을 반복하는 구조이다



통계적 언어 모델

- Statistical LM
- 말뭉치로부터 단어 시퀀스의 확률을 계산하여 문장을 생성하거나 분석하는 모델
- 대표적인 예) 마르코프 체인(Markov Chain)을 이용해 이전 단어 하나만을 기준으로 다음 단어의 확률을 예측하는 방식
- 예)
 - “안녕하세요 만나서 반갑습니다”에서 “안녕하세요 만나서”가 1000번, 그중 “안녕하세요 만나서 반갑습니다”가 700번 등장했다면
 - 확률은 700/1000로 계산되는 방식

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 구현이 간단하고 효율적이지만, 등장하지 않은 조합에 대해 확률이 0이 되는 문제, 즉 데이터 희소성(Data Sparsity) 문제가 존재한다

+)GPT와 BERT

- GPT는 OpenAI에서 개발한 언어 모델이며, 입력된 문장에 대해 다음 단어를 예측하는 자기회귀 방식의 모델이다
- BERT는 Google에서 개발한 마스킹 기반 언어 모델이며, 문장에서 일부 단어를 마스킹한 후 그 단어를 예측하는 방식이다
- GPT와 BERT는 둘 다 사전 학습(pretraining)을 통해 사전 지식을 쌓고, 다양한 자연어 처리 과제에 활용되는 대표적인 언어 모델이다

N-gram 모델

- N-gram 모델은 텍스트에서 N개의 연속된 단어 시퀀스를 기반으로 다음 단어의 출현 확률을 계산하는 통계 기반 모델이다
- N=1일 경우는 유니그램(unigram), N=2는 바이그램(bigram), N=3은 트라이그램(trigram)이라고 부른다
- N-gram 모델은 조건부 확률을 단순히 앞 N-1개의 단어로 제한하여 계산하기 때문에 구현이 쉽고 빠르다는 장점이 있다
- 하지만 문맥 길이가 제한되어 복잡한 문장 구조를 포착하기 어렵고, 희귀 단어 조합에 대한 확률 계산이 어렵다는 한계가 있다
- 예를 들어 "입이 무겁다"는 표현은 '비밀을 잘 지킨다'는 뜻이므로, N-gram을 통해 자주 등장하는 구나 연속된 단어를 분석함으로써 의미를 파악할 수 있다
- N-gram은 단어의 순서가 중요한 자연어 처리 작업에 유용하게 쓰이는 모델이다

Unigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요. 만나서. 진심으로. 반가워요

Bigram : 안녕하세요 만나서 만나서 진심으로 진심으로 반가워요 안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요

Trigram : 안녕하세요 만나서 진심으로 만나서 진심으로 반가워요 안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

TF-IDF

- TF-IDF는 문서에서 특정 단어의 중요도를 계산하는 통계적 방법이다
- BoW 방식에 가중치를 부여한 것으로, 단어의 단순 출현 여부가 아닌 중요도 반영을 목표로 한다
- BoW는 단어 출현 횟수를 기반으로 벡터화하지만, 의미 있는 단어를 구별하기 어려운 한계가 있다

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

단어 빈도

- TF: Term Frequency
- 특정 문서 내에서 특정 단어가 얼마나 자주 등장하는지를 의미한다
- TF는 단어의 출현 횟수를 기준으로 계산되며, 단어가 많이 등장할수록 높은 TF 값을 갖는다
- 수식: $TF(t,d) = \text{count}(t,d)$
- 단점은 문서 길이에 따라 TF 값이 커져 중요도가 과대평가될 수 있다는 점이다

문서 빈도

- DF: Document Frequency
- 특정 단어가 전체 문서들 중 몇 개의 문서에서 등장했는지를 의미한다

- 자주 등장하는 단어는 구별력이 낮기 때문에 DF가 높을수록 단어 중요도는 낮아진다
- 수식: $DF(t,D) = \text{count}(d \in D : t \in d)$

역문서 빈도

- IDF: Inverse Document Frequency
- 자주 등장하는 단어의 중요도를 낮추기 위해 사용된다
- 수식: $IDF(t,D) = \log(\text{count}(D) / (1 + DF(t,D)))$
- 드물게 등장하는 단어일수록 IDF가 커지며, 중요한 단어로 간주된다

TF-IDF

- TF-IDF는 단어의 중요도를 TF와 IDF의 곱으로 계산한다
- 많이 등장하지만 흔하지 않은 단어가 높은 TF-IDF 값을 가진다

수식 6.8 TF-IDF

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 주요 파라미터는 아래와 같다

input: 입력 데이터 형식 지정

encoding: 문자 인코딩 형식

lowercase: 소문자 변환 여부

stop_words: 불용어 제거 여부

ngram_range: 사용할 N-gram 범위

min_df: 최소 문서 빈도

vocabulary: 사용할 단어 사전

smooth_idf: IDF 분모를 평활화할지 여부

- 활용 방식

fit(): 학습 수행

transform(): 학습된 상태에서 데이터 변환

fit_transform(): 학습과 변환을 동시에 수행

toarray(): 결과를 배열로 출력

vocabulary_: 단어와 인덱스를 매핑한 사전 반환

- 한계점

- 문장 구조, 어순 등 맥락을 고려하지 않기 때문에 의미 파악이 어렵다
- 의미는 다르지만 단어가 같거나, 의미는 비슷하지만 단어가 다른 경우 구분이 어렵다